Chapter 3 - Backend Fundamentals

Topic	Purpose	Why We Use It	Example
ApiResponse	Standard API response	Same JSON format for all APIs	ApiResponse.success(user)
GlobalExceptionHandler	Handle all exceptions in one place	Avoid repeating try-catch in every controller	Handles 400, 403, 404, 500
ProcessMindException	Custom business exception	Clear application-specific errors	"User already exists"
Health Endpoint	Check application status	Monitoring by developers and cloud platforms	/health
Logging	Record application events	Debugging and monitoring	INFO, DEBUG, ERROR
Environment Variables	Store sensitive configuration outside code	Security and different environment settings	DB_PASSWORD, JWT_SECRET
ValidationConstants	Centralize validation rules	Avoid duplicate values	PASSWORD_MIN_LENGTH
Docker	Package application with dependencies	Runs consistently on any machine	docker-compose up
Docker Compose	Start multiple services together	Run backend + PostgreSQL with one command	docker-compose.yml
Volumes	Persist database data	Prevent data loss when containers restart	PostgreSQL data volume
Swagger	Interactive API documentation	Test APIs without Postman	/swagger-ui.html
Actuator	Application monitoring	Health and metrics endpoints	/actuator/health

Standard API Response

Java

```java
return ApiResponse.success(user);
```

Throw Custom Exception

Java

```java
throw new ProcessMindException("User already exists");
```

Global Exception Handler

Java

```java
@ControllerAdvice
public class GlobalExceptionHandler {

    @ExceptionHandler(ProcessMindException.class)
    public ResponseEntity<ApiResponse<?>> handle(ProcessMindException ex) {
        // Return standardized error response
    }
}
```

Health Endpoint

Java

```java
@GetMapping("/health")
public ApiResponse<String> health() {
    return ApiResponse.success("Application is running");
}
```

Validation Constant

Java

```java
public static final int PASSWORD_MIN_LENGTH = 8;
```

Docker

dockerfile

```dockerfile
FROM eclipse-temurin:21-jdk
COPY target/app.jar app.jar
ENTRYPOINT ["java","-jar","app.jar"]
```

Docker Compose

YAML

```yaml
services:
  postgres:
    image: postgres

  backend:
    build: .
```